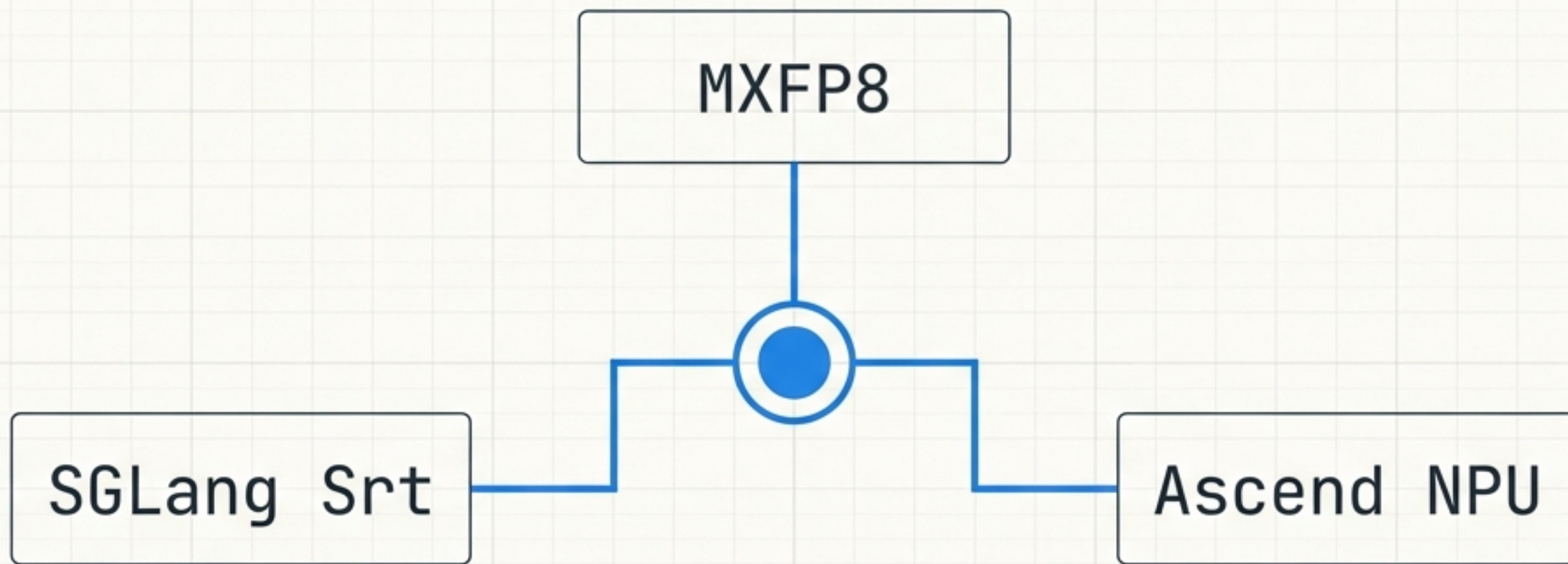


SGLang 框架 Ascend NPU MXFP8 量化适配深度解析

基于 msmodelslim 的离线适配（路线A）与在线适配（路线B）架构设计



现状分析：SGLang NPU 量化支持矩阵

量化方法 (Quantization Method)	当前状态 (Status)
W8A8 INT8 (Static/Dynamic, Linear)	✅ 已支持
W4A4 / W8A8 / W4A16 INT4/INT8 (MoE)	✅ 已支持
KV Cache 量化	🔄 进行中
MXFP8 / MXFP4	❌ 未实现



架构目标：CUDA 侧已通过 PR #17449 支持 MXFP8。本架构方案旨在补齐 Ascend 生态缺口，实现硬件层面的量化能力对齐。

底层技术基石：Ascend NPU 核心 API 支持

Ascend 硬件原语 (Hardware Primitives)

`torch_npu.npu_dynamic_mx_quant(x, dst_type)`

核心动态量化
API

`torch_npu.npu_quant_matmul(...)`

量化矩阵乘法

可行性验证：MindIE-SD 参考实现

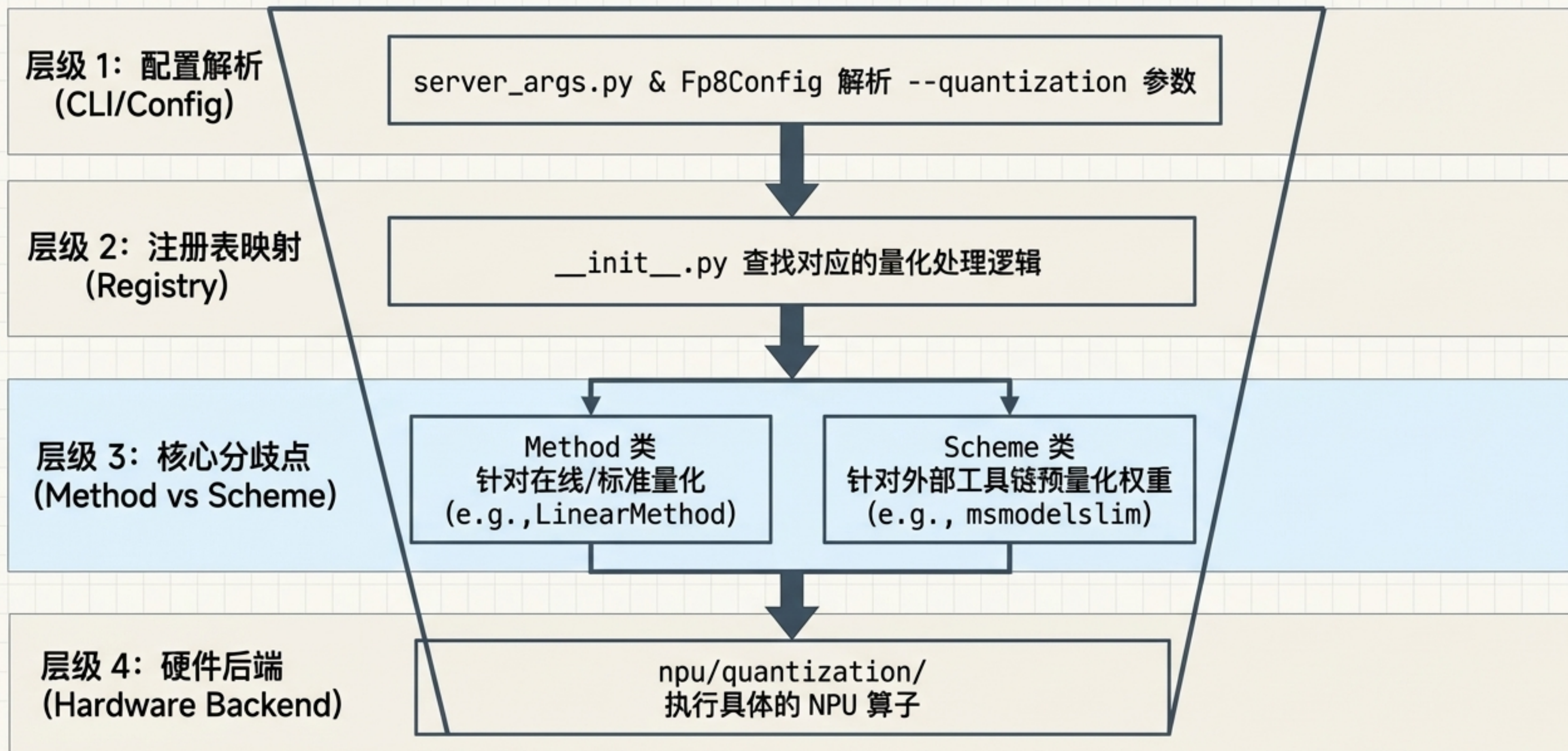
W8A8MXFP8QuantLinear

`torch_npu.npu_dynamic_mx_quant(x, dst_type)`

`torch_npu.npu_quant_matmul(...)`

一句话结论：Ascend NPU 硬件与底层算子已完全具备 MXFP8 能力，缺口仅在 SGLang 框架层的分发与绑定。

SGLang 量化框架核心机制与调用逻辑



适配双路线全景图：路线 A vs 路线 B

路线 A - msmodelslim 离线适配

- 机制：依托 SGLang 现有的 Scheme 框架，加载外部预量化权重。
- 状态：重点推荐方案（改动极小，无缝对接华为工具链）。

路线 B - Online 在线适配

- 机制：依托 Method 框架，运行时使用 `--quantization mxfp8` 动态量化。
- 状态：独立实现方案。

NPU 底层 MXFP8 算子

重点突破：路线 A (msmodelslim) 权重格式解构

msmodelslim
输出数据

组件 1: quant_model_description.json

```
1 {  
2   "model.layers.0.self_attn.q_proj.weight": "W8A8_MXFP8"  
3 }  
4
```

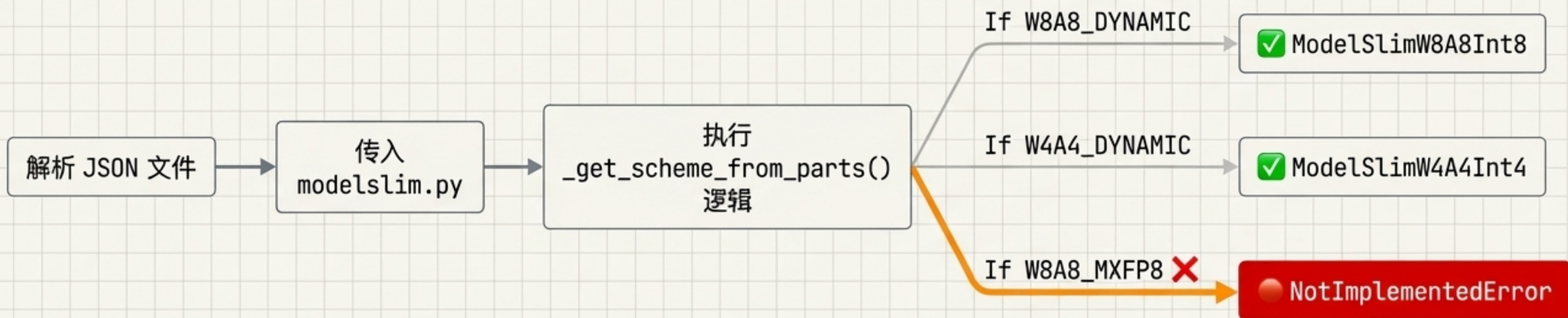
扁平字典结构，重点是拓扑指令与类型标记。

组件 2: quant_model_weight.safetensors



包含真实的物理张量数据与缩放因子。

路线 A 痛点分析：现有分发逻辑的断点



核心框架完备，仅缺乏对 MXFP8 字符串的路由分发支持。

路线 A 适配方案：Scheme 注入架构

步骤 1：创建 Scheme Node

ModelSlimMXFP8 Scheme

步骤 2：注入分发路由

```
elif type_str == 'W8A8_MXFP8':
```

自动化流转

CLI: --quantization
modelslim

读取 JSON

_get_scheme_from_parts()

W8A8_MXFP8

ModelSlimMXFP8

无缝路由：系统自动检测并流转，无需重写核心加载逻辑。

路线 A 代码级修改清单

srt/layers/quantization/

└─ modelslim/

└─ modelslim.py

└─ schemes/

└─ __init__.py

└─ modelslim_mxfp8.py

✓ 新增 (Create)

核心实现文件，承载 NPU API 调用与张量转换逻辑。

✓ 修改 (Modify)

向系统注册新的 ModelSlimMXFP8 类。

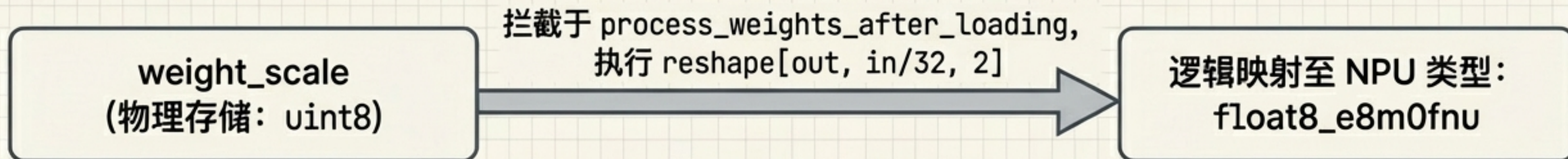
✓ 修改 (Modify)

更新 _get_scheme_from_parts 以包含 MXFP8 字符串解析分支。

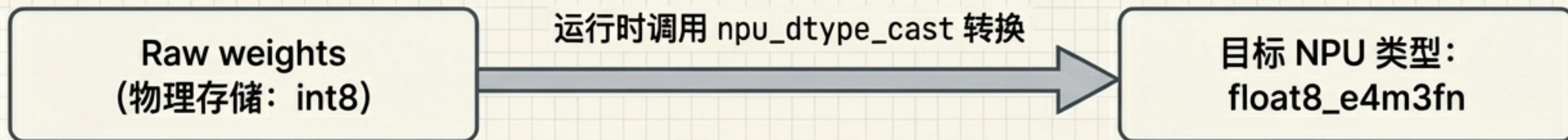
极低侵入性 (Minimal Intrusiveness) – 核心机制零破坏，仅作为插件注入。

路线 A 核心技术细节：Tensor 与 Dtype 对齐

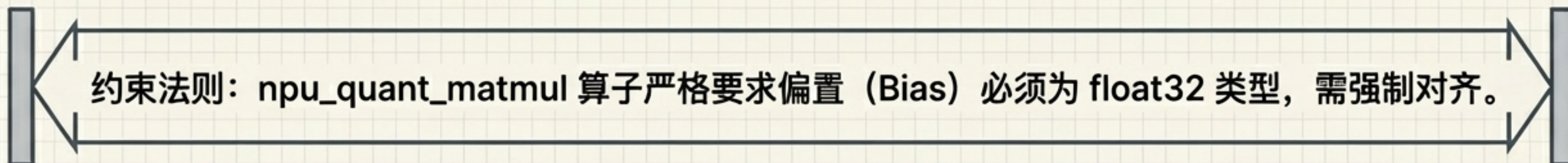
Scale Alignment (Scale 缩放因子对齐)



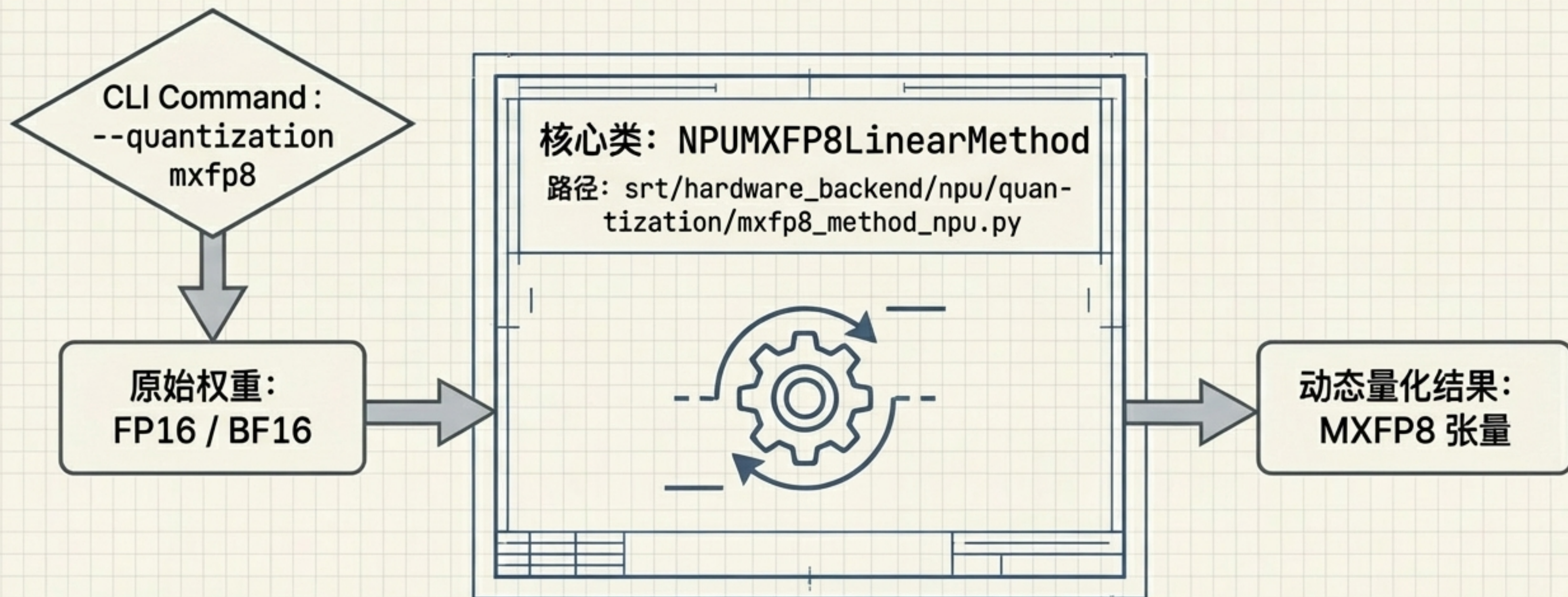
Weight Cast (权重类型转换)



Bias Cast (偏置约束)



路线 B 简述：Online MXFP8 在线量化框架



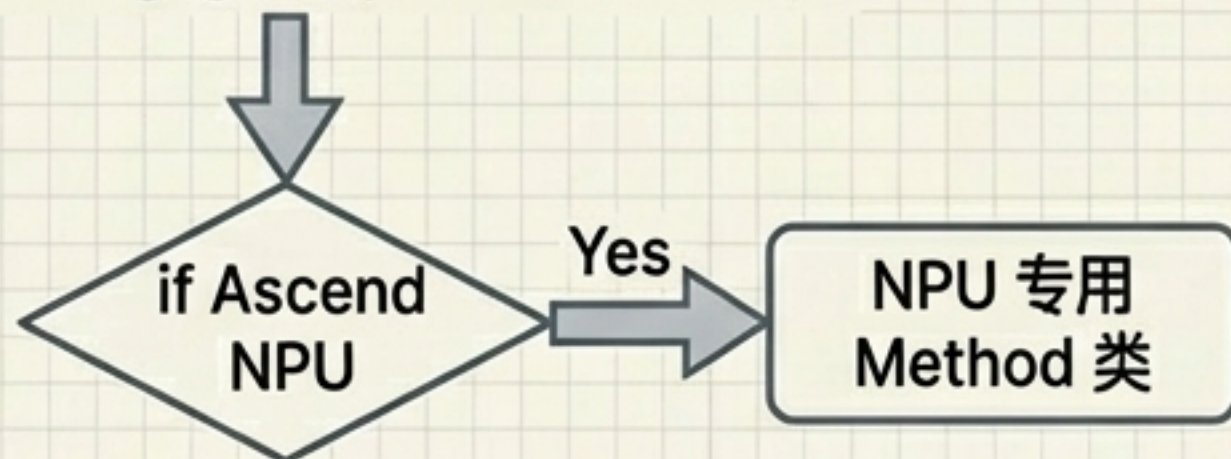
适用场景：在没有 msmodelslim 预量化权重的环境下，通过运行时动态执行精度压缩。

路线 B 简述：注册逻辑与权重加载

Hook 1: 注册逻辑 (Registration Hook)

 srt/layers/quantization/fp8.py

Fp8Config.get_quant_method()

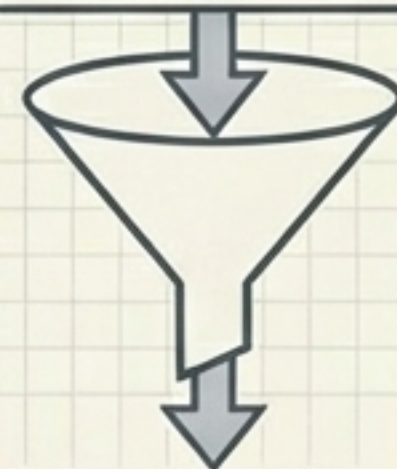


动作：在 Fp8Config.get_quant_method() 中新增 Ascend NPU 分支，拦截 fp8 请求并将其路由至 NPU 专用的 Method 类。

Hook 2: 权重加载 (Loader Hook)

 model_loader/loader.py

原始权重：
FP16 / BF16



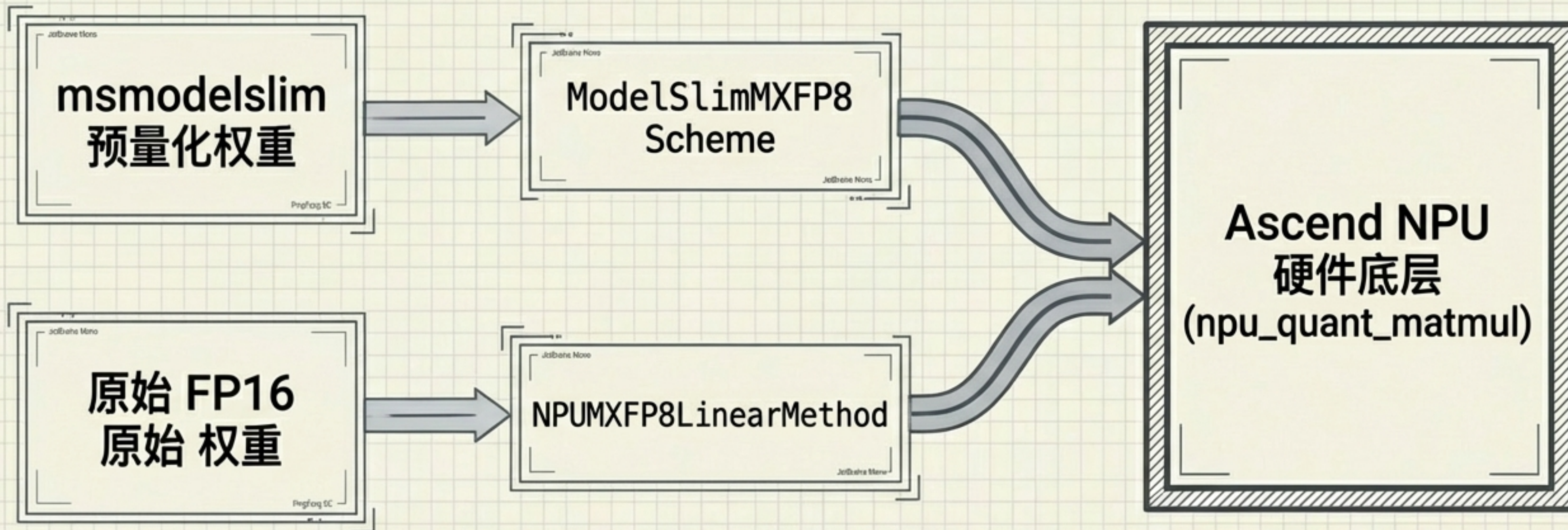
动态 downcast 至 MXFP8

动作：确保 NPU 加载路径在模型初始化阶段，具备将 FP16/BF16 动态 downcast 至 MXFP8 的处理能力。

架构对比：路线 A vs 路线 B 对比总结

维度 (Dimensions)	路线 A (Offline)	路线 B (Online)
权重来源 (Source)	msmodelslim 预量化文件	原始 FP16/BF16 模型
调用参数 (CLI)	--quantization modelslim	--quantization mxfp8
核心架构 (Core Class)	Scheme 模式拓展	Method 模式实现
数据处理介入点 (Data Hook)	加载时通过 process_weights_after_loading 处理张量	运行时动态量化或在线权重 转换

最终交付蓝图：全景闭环



结论：通过精准的代码解耦与模块化注入，SGLang 在 Ascend 硬件上实现了与 CUDA MXFP8 100% 的架构能力对齐，具备生产环境部署条件。